Project 1

Chase Kowalsky

October 25, 2026

Part A

(a) The state is the row,column pair to represent the agent's current position on the board at any given time, declared in campus.py. The initial state of the agent is the first initial row,column pair that is passed into the algorithm. The valid actions that the agent can move to is up, down, left, or right. The transition model returns the successor coordinate based off the action that was taken. The algorithm checks each state to see if it is the goal. The path costs 1 unit each time the state updates and the agent moves. Once the agent reaches the goal it recieves 39 points. BFS works best here because each move is only 1 point, meaning that the amount of moves is equal to the depth of the search. BFS checks the nodes on the current layer before moving deeper to the next. If the algorithm finds the goal on the current layer, it no longer needs to continue looking deeper, which saves cost.

(b) Manhatten distance is admissible here because adding the walls will only cause the agent to take a detour around the wall. The walls can never create a shortcut that costs less than the Manhatten distance.

(c)

Table 1: BFS and A* Search Performance

Map	**Delivery Task**	**BFS Length**	**A* Length**	**BFS Expanded**	**A* Expanded**
Baseline	S → W	[18]	[18]	[67]	[22]
Baseline	L → D	[22]	[22]	[79]	[62]
Baseline	H → P	[16]	[16]	[41]	[16]
Detour	S → W	[26]	[26]	[70]	[51]
Detour	L → D	[22]	[22]	[65]	[60]
Detour	H → P	[16]	[16]	[41]	[16]

For these tests, both algorithms reached the goal in the same length. In the tests, A* expanded less nodes for every test. BFS exapands more nodes than A* because it checks all possibilites while A* uses heuristics to expand less nodes.

1 Part B

(a) The success rate of each test was 100 percent, each time the agent reaches its destination within the step limit. The learned step directly matches the shortest path lengths found by the BFS and A* algorithms. The mean return correctly calculated the rewards for reaching the goal. These starter defaults given prove that the agent will find its way to the end regardless of the seed.

2 Part C

(b) How does changing the exploration rate affect a map that has dead ends and parallel paths to the next location?

I think that the agent will perform well in the earlier episodes and a low exploration rate but it will begin to struggle around H because of the low options the agent can make to escape the area.

The agent can go from S -¿ W in two different ways but both ways will both yield the same amount of steps. When the agent attempts to go to H, it must backtrack out the same way it came in to leave H. The agent can travel all the way around the map without hitting a deadend because it is a circle.

The only variable that I changed and experimented with was the exploration. All of the other variables remained unchanged. I changed the exploration rate to 0.05, 0.20, and 0.50

Table 2: Q-Learning Evaluation Performance Across Exploration Rates on Custom Map

ϵ Setting	Seeds Evaluated	Success Rate	Mean Steps	Mean Return
0.05	0, 1, 2	100%	17.0	23.0
0.20	0, 1, 2	100%	17.0	23.0
0.50	0, 1, 2	100%	17.0	23.0

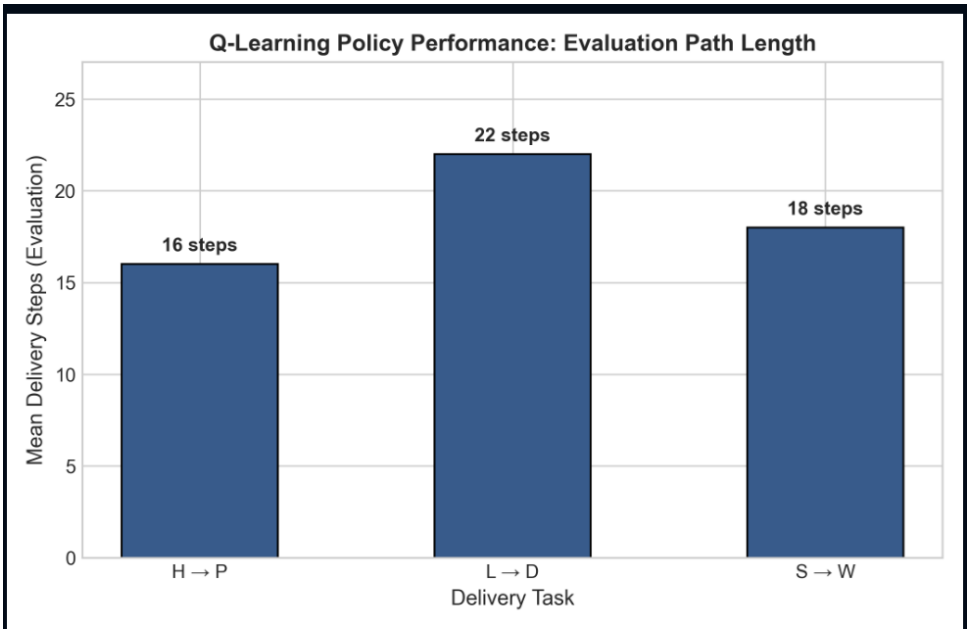


Figure 1: Evaluation path length achieved by Greedy learning. All tasks converge to optimal path lengths. (part B plot, it would not let me put it with B for some reason)

Discussion: (V) Over the course of this project I learned that having a low exploration rate does not matter if your number of training episodes is high. I experimented with 2,000 training episodes and that caused any exploration rate to achieve 100 percent success every time. Having a higher exploration rate however did cause the agent to go to the dead area where it had to backtrack to escape. I originally hypothesized that the agent would struggle in the low episodes around H because it wouldn't be able to escape. This was proven wrong because of the higher training episodes and how small the board was. It made it impossible for the agent to fail but it also made it harder on the agent to pass.